

From Features to Transformers: Redefining Ranking for Scalable Impact

Fedor Borisyyuk*
Lars Hertel*
Ganesh Parameswaran*
Gaurav Srivastava*
Sudarshan Ramanujam*
Borja Ocejo*
LinkedIn
Mountain View, CA, USA
fedorvb@gmail.com

Peng Du
Andrei Akterskii
Neil Daftary
Shao Tang
Daqi Sun
Charles Xiao
LinkedIn
Mountain View, CA, USA
pedu@linkedin.com

Deepesh Nathani
Mohit Kothari
Yun Dai
Guoyao Li
Aman Gupta
LinkedIn
Mountain View, CA, USA
dnathani@linkedin.com

Abstract

We present *LiGR*, a large-scale ranking framework developed at LinkedIn that brings state-of-the-art transformer-based modeling architectures into production. We introduce a modified transformer architecture that incorporates learned normalization and simultaneous set-wise attention to user history and ranked items. This architecture enables several breakthrough achievements, including: (1) the deprecation of most manually designed feature engineering, outperforming the prior state-of-the-art system using only few features (compared to hundreds in the baseline), (2) validation of the scaling law for ranking systems, showing improved performance with larger models, more training data, and longer context sequences, and (3) simultaneous joint scoring of items in a set-wise manner, leading to automated improvements in diversity. To enable efficient serving of large ranking models, we describe techniques to scale inference effectively using single-pass processing of user history and set-wise attention. We also summarize key insights from various ablation studies and A/B tests, highlighting the most impactful technical approaches.

CCS Concepts

• **Computing methodologies** → **Neural networks**; • **Information systems** → **Recommender systems**; **Learning to rank**.

Keywords

Large Scale Ranking, Deep Neural Networks

ACM Reference Format:

Fedor Borisyyuk, Lars Hertel, Ganesh Parameswaran, Gaurav Srivastava, Sudarshan Ramanujam, Borja Ocejo, Peng Du, Andrei Akterskii, Neil Daftary, Shao Tang, Daqi Sun, Charles Xiao, Deepesh Nathani, Mohit Kothari,

*Authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

Yun Dai, Guoyao Li, and Aman Gupta. 2018. From Features to Transformers: Redefining Ranking for Scalable Impact. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 10 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Modern professional social networks serve as digital ecosystems where users engage with a wide range of content — from job postings and industry news to social interactions and career updates. With a global user base exceeding a billion across hundreds of regions, maintaining relevance and engagement at scale is a complex challenge. Personalizing content effectively requires systems that can capture fine-grained user preferences, adapt to rapidly evolving intent, and generalize across diverse use cases.

This work is motivated by the need to evolve ranking infrastructure from traditional, feature-engineered pipelines to modern, data-driven architectures that can scale with the platform's growing complexity and user demands. Legacy systems often rely on hundreds of manually designed features, which are time-consuming to maintain and difficult to extend. Furthermore, many existing ranking models struggle to scale effectively with larger data volumes, deeper context, or larger model capacity — limiting their ability to personalize in real time.

In this paper, we propose a next-generation generative ranking framework that addresses these challenges through architectural innovation and practical scalability. Our key **contributions** are as follows:

- **Transformer-based Ranking Architecture:** We introduce a modified transformer model that incorporates learned gated normalization and set-wise attention across both user interaction history and candidate items. This design supports joint scoring of item sets, which implicitly promotes diversity in ranked outputs without requiring manual interventions.
- **Minimal Feature Dependence:** In contrast to traditional approaches that depend heavily on manually curated features and counterfactual signals, our model achieves superior ranking quality using just 7 features, significantly outperforming a strong baseline that uses several hundred. This highlights the model's capacity to learn useful representations directly from interaction data.

- **Validation of Scaling Laws:** Our architecture demonstrates effective scaling laws for ranking and retrieval systems, achieving better performance with larger models, more training data, and longer context sequences. It also outperforms the HSTU[37] and Wukong[38] baselines (see §5), establishing its superiority in ranking tasks.

To enable efficient deployment of large-scale ranking models in production, we introduce a set of inference-time optimizations, including single-pass processing over user history and set-wise attention mechanisms. These techniques are designed for scalability and low-latency inference, and are fully deployed in a real-world feed ranking system. In Section §5, we present insights from comprehensive ablation studies and online A/B tests that validate the effectiveness of our approach. The production deployment has led to measurable gains, including a 0.27% lift in Daily Active Users (DAU) engaging with professional content. We believe these contributions offer practical, battle-tested strategies for engineers and researchers working to advance large-scale ranking and retrieval systems.

2 Related Work

Deep learning systems: A variety of architectures have been proposed to improve recommendation models, beginning with the Wide & Deep model [10], which combined linear models with MLPs for memorization and generalization. Extensions explored different ways to model explicit interactions: DeepFM used factorization machines [15], DCN/DCNv2 added cross layers [33, 34], and xDeepFM introduced CIN for vector-wise feature modeling [21]. Other innovations include AutoInt with self-attention [30], AFN with logarithmic transformations [11], and FinalMLP’s dual-MLP structure [22]. InterFormer [36] unified DLRM-style and transformer-based models to jointly model engineered and temporal features. Despite these advances, most methods still rely heavily on handcrafted features. Recent efforts [6] explored unifying architectures in production, though often through trial and error. In contrast, we show that a unified transformer can outperform complex hybrid models [6], reduce reliance on counter features, and simplify development. Integrated with the LiNR GPU retrieval system [5], our approach also improves embedding-based retrieval quality.

Setwise ranking: Traditional ranking methods are point-wise, ignoring inter-item dependencies within user sessions. Set-wise approaches address this by modeling interactions among items shown together, yielding better performance [7, 13, 20, 24, 26, 28]. Transformer-based rerankers in particular have proven effective at capturing these context-driven effects.

Scaling law of recommender systems: Scaling laws describe how model performance improves with larger models, data, and compute, and are well-studied in LLMs [19]. In recommender systems, few works have examined this in industrial settings [9, 14, 37, 38]. While LLM-based recommenders have shown scaling trends [9, 14], they are hard to deploy at scale and mainly act as feature enhancers. We propose transformer modifications that outperform prior methods while using only 7 features, offering both scalability and simplicity.

3 Model Architecture

In this section, we introduce modeling approaches how we enabled ranking models on LinkedIn Feed Ranking. Within *LiGR*, a large-scale **LinkedIn Generative Recommender** ranking framework we bring state-of-the-art transformer-based modeling architectures. We introduce a modified transformer architecture that incorporates learned gated normalization and simultaneous set-wise attention to user history and ranked items. This architecture enables several breakthrough achievements, including: (1) the deprecation of most manually designed feature engineering, outperforming the prior state-of-the-art system using only 7 features (compared to hundreds in the baseline), (2) validation of the scaling law for ranking systems, showing improved performance with larger models, more training data, and longer context sequences, and (3) simultaneous joint scoring of items in a set-wise manner, leading to automated improvements in diversity. To enable efficient, production-grade serving of large ranking models, we describe techniques to scale inference effectively using single-pass processing of user history and set-wise attention.

3.1 Baseline model architecture

The baseline Feed ranking model [6] uses point-wise ranking to predict multiple action probabilities (like, comment, share, vote, long dwell, click) per <member, post> pair, combining them linearly into a final score. It employs a multi-task neural network with two towers: a Click Tower (predicting Click and Long Dwell [39]) and a Contributions Tower (predicting Likes, Comments, Shares, Votes, and an aggregated "Contribution" label). The Contribution label equals 1.0 if any of the individual actions occur. We report AUC for Contribution, Click, and Long Dwell. Both towers use the same normalized dense/sparse features [16], with ID embeddings [6] looked up in Actor and Hashtag tables [23]. A model architecture diagram is included in [6] for reproducibility.

3.2 Generative Ranking with Setwise attention and Learnt normalization

LiGR, our proposed large-scale ranking framework is fully sequential similar to the generative recommender (GR) approach described in [37]. Specifically, the model input is a member’s interaction history $X_0, \dots, X_n$, where each interaction X_i represents one token. An input token X_i is itself a concatenation of multiple embedding features which represent entities such as the item, the creator of the item, or content embeddings. The model output is a sequence $y_0, \dots, y_n$ which is the sequence of actions corresponding to the inputs. For example, y_i would be a multi-hot vector representing all the actions that were taken on item i by the member. Here, actions can be click, like, comment, share etc. Using [37]’s action interleaving scheme, we interleave inputs with their corresponding outputs (see Figure 1). This allows the model to combine historical item features with member actions to learn rich internal feature representations. Causal attention prevents position i to look ahead at its outputs. The model outputs corresponding to interleaved actions are masked. The model consists of Transformer blocks [31] with pre-norm [2, 12, 32]. Each multi-head attention and feed-forward layer is gated with a linear projection and sigmoid activation: $h^{j+1} = h^j + F(h^j) \times \sigma(h^j W)$, where F represents the

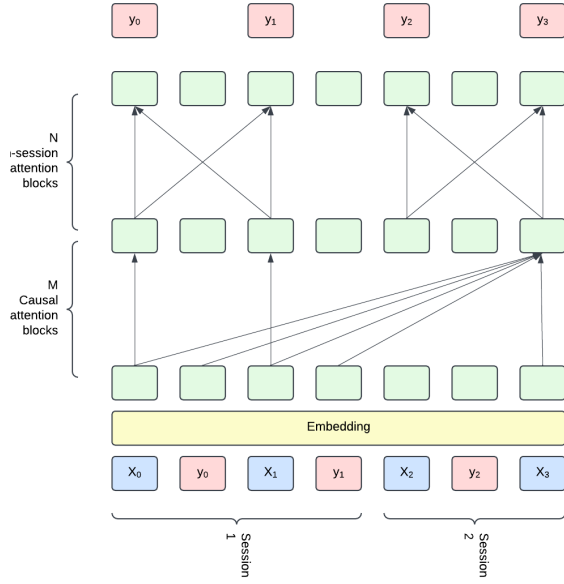


Figure 1: LiGR architecture combining historical attention for feature aggregation and in-session attention.

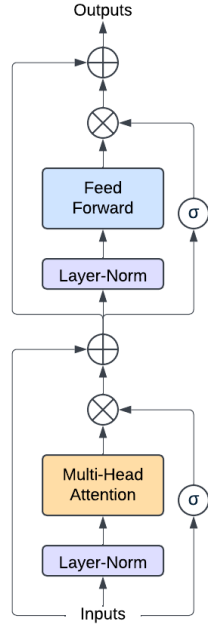


Figure 2: LiGR Transformer architecture using a gating skip-connection.

multi-head attention and feed-forward layer as shown in Figure 2. Proposed architecture has similarity to Highway transformer [8], but instead of applying gating on skip connections we apply it on output of multi-head attention and Feed Forward layer.

Setwise Ranking Layer: The LinkedIn Feed ranking model has historically relied on a pointwise approach with rule-based diversity re-rankers. These re-rankers enforce session-level policies, such as: (1) a minimum gap of two items between out-of-network content, (2) a minimum gap of two items between posts by the same actor. These rules assume a one-size-fits-all solution, which can be limiting. To improve, we propose adding a setwise model layer to re-rank items within the session by leveraging user interactions throughout the session and utilizing session-level data instead of treating each item independently on a point-wise basis. These changes aim to enhance session experiences and increase user engagement with the LinkedIn Feed. We further extend the *LiGR* model to allow for session-level information flow similar to the SetRank architecture proposed in [24]. SetRank introduces self-attention between items in a session to facilitate list-wise ranking. Specifically, we augment the *LiGR* model with in-session attention blocks as shown in Figure 1. Due to the fact, that historical sessions are of varying length, in-session attention requires an attention mask that varies depending on the session ID inputs. We achieve this efficiently using FlexAttention [17]. Finally, we augment the binary cross-entropy training loss using the Attention Rank loss [1]. Again, aggregation in the Attention Rank loss happens on a session level in our case, determined by session IDs.

3.3 LiGR with Semantic IDs

One of the challenges with ID-based models is the need to manage large vocabularies of sparse ID features. This leads to increased model sizes, making model serving more complex and requiring continuous updates to keep IDs fresh. For instance, new posts are created by LinkedIn members every second. Large models with trillions of parameters often demand sophisticated distributed serving infrastructure [37], where different parts of the model are hosted across multiple machines. Recent research demonstrates that using semantic IDs [29] can significantly reduce model size while maintaining the same level of performance. In §5.1, we show that this approach allows us to reduce the model size from 5.4B to 1.3B parameters without compromising model quality. We investigate the use of Semantic IDs as additional features by leveraging the RQ-VAE model to generate discrete hierarchical IDs through unsupervised learning, as proposed in [27] and [29]. The residual-quantized variational auto-encoder (RQ-VAE) model receives as input a batch of a high-dimensional feature vectors x and assign to each item a tuple of semantic IDs. As the first step the encoder network f_ϕ downscale the input embedding x to a latent lower-dimensional representation $z = f_\phi(x)$ where f_ϕ is the encoder network parameterized by ϕ and $z \in \mathbb{R}^d$ is the latent representation. The latent representation z is quantized using hierarchical residual quantization (RQ). This involves iteratively quantizing z using a series of codebooks:

- Step 1: Initial Quantization. The first quantized code c_1 is obtained by finding the nearest vector in the first codebook C_1 to z , where $c_1 = \text{Quantize}(z, C_1)$, $c_1 \in C_1$. The residual r_1 is a difference between z and c_1 : $r_1 = z - c_1$.
- Step 2: Residual Quantization (Iterative). The residual r_1 is quantized iteratively using subsequent codebooks $C_2, C_3, \dots$. At each step i : $c_i = \text{Quantize}(r_{i-1}, C_i)$, $c_i \in C_i, r_i = r_{i-1} -$

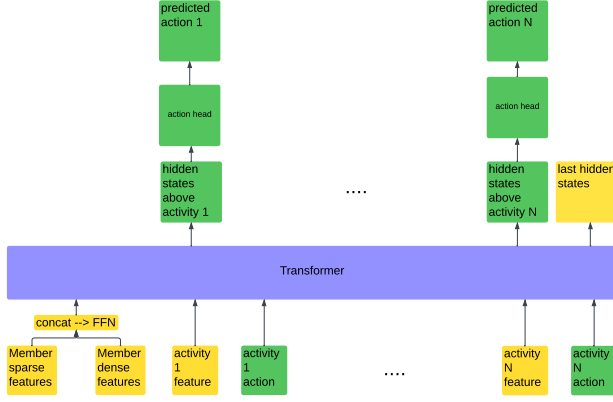


Figure 3: LiGR member tower based on LiGR. Member profile features and member activity history are inputs to a transformer model, green blocks are used when LiGR is enabled.

c_i , where c_i is the quantized code at the i -th step, r_i is the residual after subtracting c_i .

- **Final Quantized Representation.** The final quantized latent representation z_q is the sum of all quantized codes: $z_q = c_1 + c_2 + \dots + c_N = \sum_{i=1}^N c_i$

The decoder g_θ maps the quantized representation z_q back to the original embedding space, reconstructing $\hat{x} = g_\theta(z_q)$, where g_θ is the decoder network parameterized by θ , $\hat{x}$ is the reconstructed embedding.

Loss Functions: the RQ-VAE is trained by minimizing the following losses: (1) Reconstruction Loss, which measures the difference between the input embedding x and the reconstructed embedding $\hat{x}$: $\mathcal{L}_{\text{reconstruction}} = \|x - \hat{x}\|^2$; (2) Quantization Loss, which penalizes the difference between the latent representation z and its quantized version z_q : $\mathcal{L}_{\text{quantization}} = \|z - z_q\|^2$. The total loss is a weighted combination of the reconstruction and quantization losses: $\mathcal{L} = \mathcal{L}_{\text{reconstruction}} + \beta \mathcal{L}_{\text{quantization}}$, where β is a hyperparameter controlling the importance of quantization losses. We use $\beta = 0.25$.

To stabilize training, we use vector reset approach suggested in [29, 35]. We track exponential moving averages of number of assignments for every codebook vectors. In case codebook vectors has utilization below threshold we reset it by a randomly sampled content embedding element from the current batch. The best trained RQ-VAE model includes 3 codebooks, with 1,000 code vectors each. Dimension of latent vectors is 8. To force the RQ-VAE model to learn to ensure evenly distributed clusters we apply FLOPs Regularizer [25]. After training, we perform inference to assign each item a tuple of Semantic IDs based on content features, then respective Semantic IDs embeddings are learnt in embedding bag as part of the LiGR model.

3.4 Retrieval with LiGR

We also conducted experiments on video retrieval tasks using LiGR. In the member tower (Figure 3), we embed the member's sparse features, concatenate them with dense features, and pass them

through a feedforward network (FFN). The FFN output is treated as a virtual token for a transformer model. Additional tokens for the transformer include embeddings of activities from the member's history. The baseline does not include the "green blocks," which are activated only when generative recommendation (GR) is enabled. Under GR, a shared action head predicts the next action for an activity based on the member profile, prior activities, and their corresponding actions.

In the item tower, item features are treated as individual tokens, and a transformer model processes these tokens. The LiNR model [5] is then used to calculate the similarity between member embeddings and item embeddings. The baseline loss function is a softmax loss with one positive example and two mined negative examples per training data point. For LiGR, the loss combines the two-tower softmax loss with the action prediction loss, which is itself a softmax loss predicting the correct action among all possible actions for the activities.

4 System Architecture

LiGR helps to significantly simplify our system. As we are able to reduce dependency on the counter features, and reduce number to less than ten features from hundreds. Serving of transformer model requires to store near-line member and item activity as shown on the Figure 4. Within the system we utilize variety of item and member embeddings generated by GNN [4], LLM [3], as well as ID embeddings stored and served within the model. Given the user request, the system retrieves the top candidates using the model-based retrieval system [5], then the top K candidates are evaluated using the second layer LiGR scoring model. LiGR takes as input Items features and Member Context.

4.0.1 LiGR training. Training of the LiGR model is GPU memory intensive, requiring us to leverage several optimizations. Firstly, long user item histories interleaved with corresponding actions result in sequence lengths of up to 2048 for our largest model. We used Flash Attention and mixed precision to reduce memory consumption.

Secondly, LiGR leverages several ID embedding features as shown in Table 2. It is important to scale up ID embedding dimensions alongside other hyper-parameters. In order to accommodate embedding tables larger than the memory of one GPU we use column-wise splitting of embedding tables and distribution across multiple devices. The latter allowed us to scale up embedding tables to arbitrarily large sizes.

4.0.2 Optimization of inference. Naive inference can be expensive in the LiGR model due to the complexity of applying many transformer layers on a long user action history for many candidate items. [18, 37] provide ways to amortize the computation of the history across all candidate items. In our case, the historical attention implementation allows items to attend to other items only if they are from earlier sessions. For this reason we can easily infer all candidates at the same time, automatically amortizing the inference of the history across all candidates.

Feed metrics are sensitive to latency, requiring careful optimizations to stay within bounds: (1) Split Scoring: Setwise scoring was applied only to the top 10 posts after point-wise scoring, where

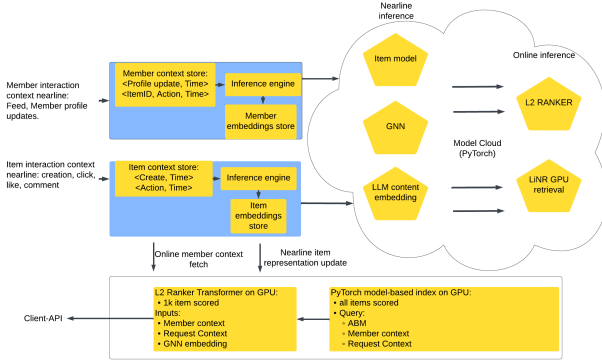


Figure 4: *LiGR* system architecture.

we decompose point-wise and setwise model weights separately; (2) Score Combination: Setwise scores were added to pointwise scores for items 1–10, preserving the order for items 11–end; (3) Simplification: Rule-based diversity rerankers were disabled; (4) Unified Inference: Pointwise and setwise parts of the model were stitched with a rewriter, enabling single inference to score the entire model. To scale online scoring, all items were passed using the batch size dimension, repackaged as a listwise dataset, and scored via the trained model. These optimizations reduced incremental latency cost to 10ms (p90), ensuring minimal impact on member experience.

5 Experiments

We conduct offline ablation experiments across Feed Ranking and Video Recommendation surfaces, and report A/B test for deployment of Setwise part of the model. In Feed Ranking, we rely on AUC metrics, which have shown a correlation with production online A/B test results.

5.1 Ranking

Scaling law in *LiGR*: We perform multiple experiments to observe scaling law in *LiGR*. Figure 5, 6, 7 and 8 demonstrate how evaluation loss, long dwell AUC and contributions AUC scale with log of the training flops. The figures show that for every order of magnitude increase in training FLOPS, the evaluation Long Dwell AUC improves by approximately 0.015, demonstrating a consistent positive scaling effect. Figure 8 expresses these experiments in terms of number of dense parameters and sequence length. We scaled the model to include 5.4 billion sparse ID embedding parameters, utilizing 64-dimensional ID embeddings with a vocabulary size of 33 million, implemented using a collision-based embedding bag. The model also incorporates 10 million dense parameters across 16 layers of a standard Transformer architecture, enhanced with dense gating on residual connections. Historical sequences were scaled to include up to 1,024 feed interactions per member over a six-month period. Training was performed on 8 A100 GPUs using 110 million training sequences across January to August 2024, and evaluation was conducted on 6 hours of data immediately following the training period, covering approximately 4 million samples. Performance

metrics for the largest scaled model are provided in Table 1. We find that *LiGR* leads to a 2.4% increase in Long Dwell AUC and a 1.2% increase in Contributions AUC. Each action is modeled as a separate task. The Long Dwell task is defined in [39].

Scaling law of *LiGR* vs HSTU [37]: We experimented by replacing *LiGR* transformer layers (Figure 2) with HSTU layer. Wherein, we observed performance drop when using HSTU layers. Figure 5, 8, 6 and 7 demonstrate how HSTU underperforms compared to *LiGR* across metrics such as normalized entropy, Long dwell AUC and Contributions AUC. Training FLOPS on the Figures are an approximate function of the number of dense parameters times the sequence length. For example, at the same compute (10^{17} flops) the long dwell AUC drops from 76.03 for *LiGR* to 75.87 when using HSTU. Additionally, we were able to train larger models with the *LiGR* transformer layers. This is because we could use FlashAttention to optimize its memory consumption.

Scaling law of Wukong: We explored model scaling using Wukong [38], which scales dense parameters to capture higher-order interactions. However, applying it to Feed models showed no AUC gains, even with 8 layers. We suspect this is due to information loss from mixing heterogeneous feature types—numerical and categorical—during deep interactions.

Effect of sequence length, number of layers, and ID embedding dimension: [37] noted that *LiGR* scaling laws depend on the scaling of sequence length, number of transformer layers, and ID embedding dimension in parallel. To demonstrate scaling in Figures 5, 8, 6, 7 we followed a similar approach. However in this section, we discuss what happens when we scale each dimension individually. Figures 9, 10, and 11 show AUC and evaluation loss when scaling sequence length, number of transformer layers, or ID embedding dimension while holding everything else constant.

From these plots we make the following observations about scaling laws when model capacity is scaled in isolation along a specific dimension: ID embedding dimension, number of transformer layers, and sequence length. Increasing the sequence length consistently demonstrates clear scaling laws across all evaluation metrics, including Evaluation Loss, Long Dwell AUC, and Contributions AUC. This suggests that longer sequences effectively leverage the model’s capacity for improved performance. Similar observation can be made from Figure 8. In contrast, independently increasing the ID embedding dimension or the number of transformer layers does not consistently demonstrate scaling laws for Evaluation Loss. While some metrics, such as Long Dwell AUC, show an upward trend with scaling, others, like Contributions AUC, exhibit irregular or plateauing behavior. These observations suggest that to fully realize scaling laws across all metrics, it is necessary to scale multiple dimensions simultaneously rather than in isolation.

***LiGR* Feature Ablation:** We conducted feature ablation in two ways. First, we evaluated which individual feature achieved the highest AUC on its own (Table 2). Second, we analyzed which feature performed best when combined with content embeddings (Table 3). On LinkedIn Feed, the relationship between the viewer and the actor (the author of the post) is important. As expected, Actor IDs were the significant feature in both ablation studies. For efficiency, feature ablations were conducted on smaller model configurations, using 12 Transformer layers, a sequence length of

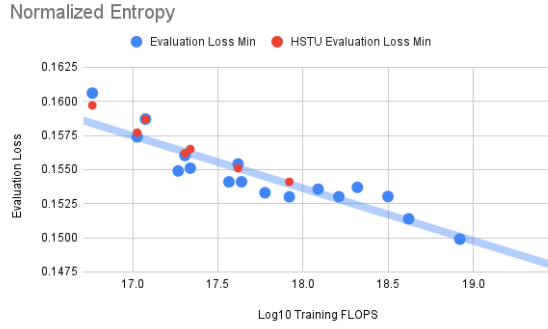


Figure 5: Scaling of normalized evaluation entropy as a function of training FLOPS for *LiGR* and HSTU.¹

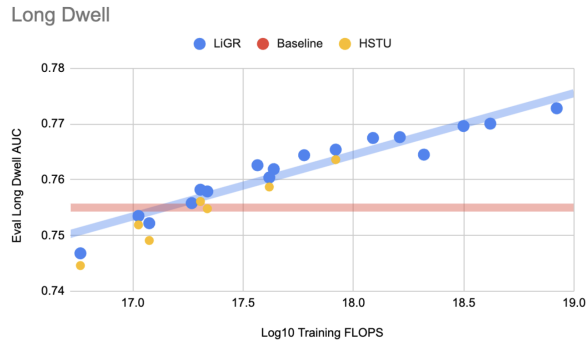


Figure 6: Scaling of Long Dwell AUC as a function of training FLOPS for *LiGR* and HSTU.¹

512, and an ID embedding dimension of 32. As a result, the metrics reported in Tables 2 and 3 are lower than those in Table 1.

***LiGR* with In-session Setwise Attention:** Table 4 compares AUC between a *LiGR* model with different attention mechanisms. One model uses historical attention only. The other uses both historical attention and in-session setwise attention. The table shows that providing the model with in-session attention results in an additional 0.2% Long Dwell AUC gain.

Model	Long Dwell AUC	Contributions AUC
Baseline	0.755	0.903
<i>LiGR</i>	0.773	0.914
Difference	+2.4%	+1.2%

Table 1: Model Performance Comparison

***LiGR* with Semantic IDs:** In the experiments with Semantic IDs we wanted to investigate if Semantic IDs could be used to replace Post ID (database assigned IDs to posts). We train a Residual-Quantized Variational Autoencoder on post embeddings to generate semantic IDs (SIDs) for each post using three hierarchical codebooks, each with 1,000 dimensions.

¹We don't show HSTU points beyond value of 18 as the open source code of HSTU went out of memory for larger model configurations. Training FLOPS are an approximate function of the number of dense parameters times the sequence length.

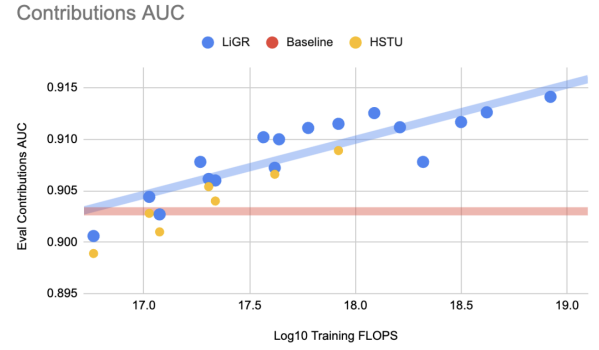


Figure 7: Scaling of Contributions AUC as a function of training FLOPS for *LiGR* and HSTU.¹

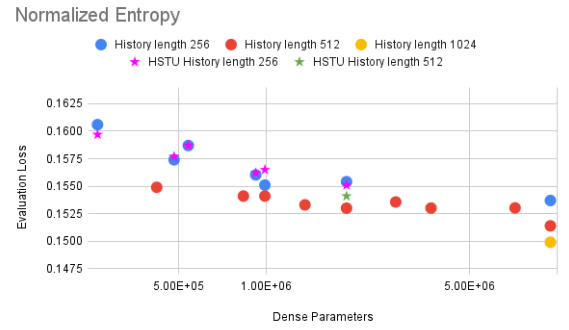


Figure 8: Scaling of normalized evaluation entropy as a function of training FLOPS for *LiGR* and HSTU for different sequence lengths.

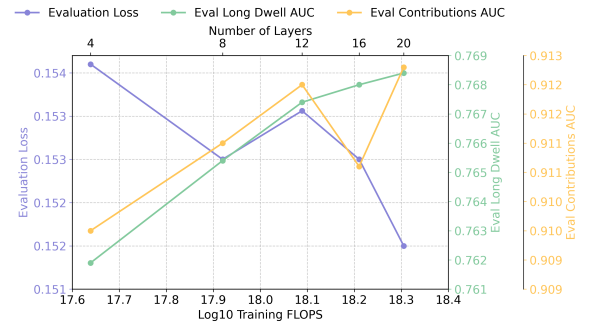


Figure 9: Scaling number of transformer layers while keeping all other hyper parameters constant (id embedding dimension = 32, sequence length = 512).

For SIDs, we adopt a prefix encoding approach: (1) first-level SIDs: these are directly retrieved from the embedding bag as learned during training; (2) bi-gram representations: the first and second-level IDs are concatenated and mapped to their corresponding vectors in the embedding bag learned during training; (3) tri-gram representations: the concatenation of first, second, and third-level

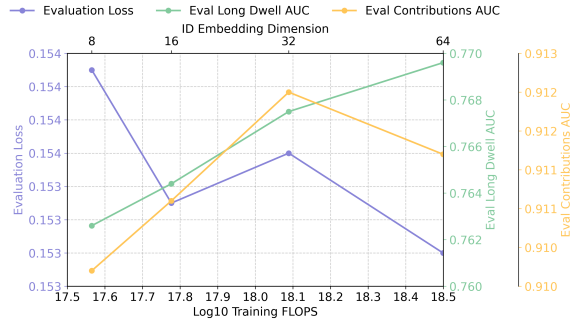


Figure 10: Scaling id embedding dimension while keeping all other hyper parameters constant (number of transformer layers = 12, sequence length = 512).

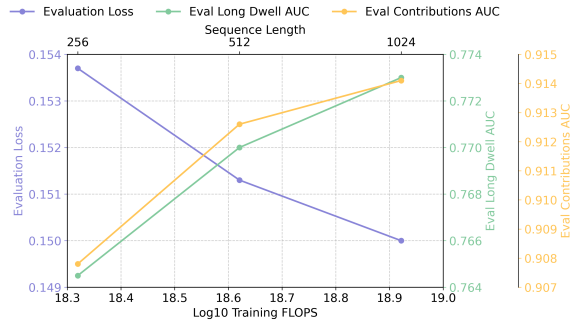


Figure 11: Scaling sequence length while keeping all other hyper parameters constant (number of transformer layers = 16, id embedding dimension = 64).

Single feature	Long Dwell	Contribution
Post ID	0.703	0.857
Original Actor ID	0.731	0.893
Post Type (Video/Photo/..)	0.706	0.883
Update Age of Post	0.680	0.854
Activity ID of Shared Post	0.695	0.856
Post content embedding[3]	0.703	0.878
All features	0.767	0.912
Baseline	0.755	0.904

Table 2: AUC of models trained with only one feature.

LiGR Features	Long Dwell	Contribution
PE + Actor IDs	0.755	0.911
PE + Post Type (Video/Photo/..)	0.741	0.897
PE + Item IDs	0.752	0.895
All features	0.767	0.912
Baseline	0.755	0.904

Table 3: Comparison of AUC of models trained with post content embeddings (PE) [3] and one additional feature.

IDs is hashed into an embedding bag of size 10^6 . We use 32 dimensional embedding for each SID to learn representation for prefix encoding. We use concat pooling for unigram, bi-gram and tri-gram SID embeddings in the *LiGR* model. We observe the AUC lift by including SID embeddings for both long dwell and contribution

Attention	Click	Long Dwell	Contribution
Difference	+0.50%	+0.20%	-0.04%

Table 4: Relative AUC improvement for history attention only compared to history and in-session attention.

objectives. We observe that Post ID feature could be safely removed and replaced by SID.

Feature	Long Dwell	Contribution
All features (Table 2)	-	-
Only SID	-6.5%	-4.6%
Remove Post ID, Add SID	+0.4%	+0.11%
Add SID	+0.26%	+0.22%

Table 5: Performance comparisons with SIDs prefix encoding.

5.2 LiGR for Retrieval

We experimented with in house video retrieval. We used a time range of 2 weeks worth of data. The retrieval system incorporated *LiGR* into LinkedIn’s PyTorch GPU-based retrieval system, LiNR [5], and tested two retrieval modes: cosine similarity distance and an MLP-based model. In the MLP model, member and video embeddings were concatenated, and a fully connected layer was applied to learn the interaction score. We explored settings where the transformer layers between the member tower and item tower are shared and if the ID of the items are added. We allocated 6 layers for item transformer layers, and additional 6 layers for Member transformer. If sharing layer is enabled, then the 6 item transformer layers will be shared both for item representation of candidates and for item representation within history of the member model. From the experiments we can see that sharing layers, using MLP model and *LiGR* as well as using video ID boosted the recall@400. The experiments results are an average of 3 duplicated runs for each setting.

Experiment set up	Recall@400	Relative Lift to Row Above
Baseline: No <i>LiGR</i> , not sharing layers, cosine similarity LiNR model	0.0799	–
No <i>LiGR</i> , sharing layers, cosine similarity LiNR model	0.1197	49.81%
No <i>LiGR</i> , sharing layers, MLP LiNR model	0.3905	226.23%
With <i>LiGR</i> , sharing layers, MLP LiNR model	0.3997	2.36%
With <i>LiGR</i> , sharing layers + Video ID, MLP LiNR model	0.4435	10.96%

Table 6: Retrieval LiGR measurements.

5.3 A/B tests

Model Description	DAU	Time Spent on Feed
Setwise with Diversity Rules Disabled	+0.27%	+0.28%

Table 7: Online Ramp of Setwise part of LiGR

The ramp results for setwise part of the model show that incorporating a model to account for list-level interactions after pointwise ranking leads to a 0.27% increase in the number of daily active users engaging with professional content on LinkedIn, as well as an overall increase in the time spent on the feed. To optimize latency, the setwise and pointwise components of the model are separated. In online experiments, the setwise layer is applied only to the top 10 posts during inference.

Furthermore, with this new modeling layer in place we plan to optimize for session level metrics next (e.g. time to next session, number of impressions in session), which was not possible previously with a pure point-wise approach.

6 Lessons learnt

Over the time of development of *LiGR* we learnt many lessons. Here we present couple of interesting examples.

6.1 Diversity of topics in Recommendations

Over the years, the LinkedIn feed ranking model has been a pure pointwise model and list level interactions are not taken into account by the model. This could however yield a subpar experience of the entire feed session as a whole for our members. Some examples include seeing too many updates from the same actors, back to back out of network content, back to back instances of posts that are surfaced because one of their connections liked an activity of their connection.

Diversifying this experience through a rule based approach has helped provide a much better session experience as a whole for our members which could be seen from the metric impact. We did a simple ablation study of removing all the diversity rules and checked the member impact. As expected, we do see a drop in the DAU in LinkedIn (-0.18%).

While the rule based diversity for organizing the session does help, we believe it is suboptimal and a model powered solution could yield a better experience for our members. The rules tend to assume that the same template would work for everyone. In our approach we believe that replacing this legacy solution with a model that could learn the required list level diversity attribute is a superior solution. In fact a model based approach could help us quantify and keep diversity as an objective in the longer term. In this work, we show how setwise models could replace diversity rules with a superior member experience in LinkedIn.

6.2 Our approach to develop *LiGR*

LiGR and the traditional DLRM approach are fundamentally different in data format, features, and model training. *LiGR* therefore required a full rebuilding of our training pipeline. The goals for this rebuilding were to use few features, to outperform the baseline, and to demonstrate scaling laws.

We started this work by verifying that our most important count features can be emulated by ID embedding features alone. Then we built a small *LiGR* model and added the top features from the DLRM model or corresponding ID feature counterparts until adding more features gave only small AUC improvements. While the resulting model had ten times less features than the baseline, we did find that with too few features the model can suffer from item cold start and

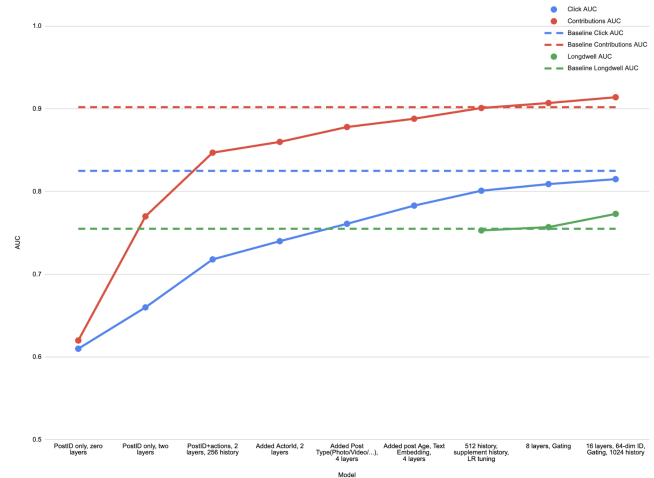


Figure 12: GR AUC improvement over successive iterations.

is not able to beat the baseline. In order to reduce member cold start in less frequent members we also increased the user history time window, but retained item ID features only for a limited window.

Having added enough features, we scaled up the model along the dimensions of sequence length, embedding dimension, and number of layers. During this phase the main challenges were to maintain training stability and manage GPU memory. To ensure stable training we used different learning rates for dense vs. sparse model parameters, dense gating of transformer layers, and transformers with gating as shown in Figure 2.

Overall we found that aside from ensuring training stability, most architecture changes we experimented with had little impact on the prediction AUC. The majority of performance appeared to be driven by features and model scale.

7 Conclusion

In this paper, we introduced the *LiGR* framework, encapsulating our experience in developing state-of-the-art models. We discussed various modeling architectures and their combination to create a high-performance model for delivering relevant user recommendations. The insights shared in this paper can benefit practitioners across the industry.

References

- [1] Qingyao Ai, Keping Bi, Jiafeng Guo, and W Bruce Croft. 2018. Learning a deep listwise context model for ranking refinement. In *SIGIR*.
- [2] Alexei Baevski and Michael Auli. 2018. Adaptive input representations for neural language modeling. *arXiv preprint arXiv:1809.10853* (2018).
- [3] Akanksha Bindal, Sudarshan Ramanujam, Dave Golland, TJ Hazen, Tina Jiang, Fengyu Zhang, and Peng Yan. 2024. LiPost: Improved Content Understanding With Effective Use of Multi-task Contrastive Learning. *arXiv:2405.11344* [cs.LG] <https://arxiv.org/abs/2405.11344>
- [4] Fedor Borisyyuk, Shihai He, Yunbo Ouyang, Morteza Ramezani, Peng Du, Xiaochen Hou, Chengming Jiang, Nitin Pasumarthy, Priya Bannur, Birjodh Tiwana, Ping Liu, Siddharth Dangi, Daqi Sun, Zhoutao Pei, Xiao Shi, Sirou Zhu, Qianqi Shen, Kuang-Hsuan Lee, David Stein, Baolei Li, Haichao Wei, Amol Ghoting, and Souvik Ghosh. 2024. LiGNN: Graph Neural Networks at LinkedIn. *KDD*.
- [5] Fedor Borisyyuk, Qingquan Song, Mingzhou Zhou, Ganesh Parameswaran, Madhu Arun, Siva Popuri, Tugrul Bingol, Zhuotao Pei, Kuang-Hsuan Lee, Lu Zheng, Qizhan Shao, Ali Naqvi, Sen Zhou, and Aman Gupta. 2024. LiNR: Model Based Neural Retrieval on GPUs at LinkedIn. *CIKM*.

- [6] Fedor Borisov, Mingzhou Zhou, Qingquan Song, Siyu Zhu, Birjodh Tiwana, Ganesh Parameswaran, Siddharth Dangi, Lars Hertel, Qiang Charles Xiao, Xiaochen Hou, Yunbo Ouyang, Aman Gupta, Sheallika Singh, Dan Liu, Hailing Cheng, Lei Le, Jonathan Hung, Sathya Keerthi, Ruoyan Wang, Fengyu Zhang, Mohit Kothari, Chen Zhu, Daqi Sun, Yun Dai, Xun Luan, Siyuan Zhu, Zhiwei Wang, Neil Daftary, Qianqi Shen, Chengming Jiang, Haichao Wei, Maneesh Varshney, Amol Ghoting, and Souvik Ghosh. 2024. LiRank: Industrial Large Scale Ranking Models at LinkedIn. KDD.
- [7] Maarten Buyl, Paul Missault, and Pierre-Antoine Sondag. 2023. RankFormer: Listwise Learning-to-Rank Using Listwise Labels. KDD.
- [8] Yekun Chai, Shuo Jin, and Xinwen Hou. 2020. Highway Transformer: Self-Gating Enhanced Self-Attentive Networks. In *PACL*.
- [9] Junyi Chen, Lu Chi, Bingyue Peng, and Zehuan Yuan. 2024. HLLM: Enhancing Sequential Recommendations via Hierarchical Large Language Models for Item and User Modeling. arXiv:2409.12740 [cs.LG] <https://arxiv.org/abs/2409.12740>
- [10] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishikesh Aradhya, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Isir, and et al. 2016. Wide & Deep Learning for Recommender Systems. *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems* (2016). <https://doi.org/10.1145/2988450.2988454>
- [11] Weiyu Cheng, Yanyan Shen, and Linpeng Huang. 2019. Adaptive Factorization Network: Learning Adaptive-Order Feature Interactions. *ArXiv abs/1909.03276* (2019). <https://api.semanticscholar.org/CorpusID:202539143>
- [12] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating long sequences with sparse transformers. *ArXiv preprint arXiv:1904.10509* (2019).
- [13] Yufei Feng, Binbin Hu, Yu Gong, Fei Sun, Qingwen Liu, and Wenwu Ou. 2021. GRN: Generative Rerank Network for Context-wise Recommendation. arXiv:2104.00860 [cs.LG] <https://arxiv.org/abs/2104.00860>
- [14] Hamed Firooz, Maziar Sanjabi, Adrian Englhardt, Aman Gupta, Ben Levine, Dre Olgiati, Gungor Polatkan, Iuliia Melnychuk, Karthik Ramgopal, Kirill Talanin, Kutta Srinivasan, Luke Simon, Natesh Sivasubramaniapillai, Necip Fazil Ayan, Qingquan Song, Samira Sriram, Souvik Ghosh, Tao Song, Tejas Dharamsi, Vignesh Kothapalli, Xiaoling Zhai, Ya Xu, Yu Wang, and Yun Dai. 2025. 360Brew: A Decoder-only Foundation Model for Personalized Ranking and Recommendation. arXiv:2501.16450 [cs.LG] <https://arxiv.org/abs/2501.16450>
- [15] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: A Factorization-Machine based Neural Network for CTR Prediction. *ArXiv abs/1703.04247* (2017). <https://api.semanticscholar.org/CorpusID:970388>
- [16] Malay Haldar, Mustafa Abdool, Prashant Ramanathan, Tao Xu, Shulin Yang, Huizhong Duan, Qing Zhang, Nick Barrow-Williams, Bradley C. Turnbull, Brendan M. Collins, and Thomas Legrand. 2019. Applying Deep Learning to Airbnb Search. KDD.
- [17] Horace He, Driss Guessous, Yanbo Liang, and Joy Dong. 2024. FlexAttention: The Flexibility of PyTorch with the Performance of FlashAttention. <https://pytorch.org/blog/flexattention/>
- [18] Lars Hertel, Neil Daftary, Fedor Borisov, Aman Gupta, and Rahul Mazumder. 2024. Efficient user history modeling with amortized inference for deep learning recommendation models. arXiv:2412.06924 [cs.LG] <https://arxiv.org/abs/2412.06924>
- [19] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. arXiv:2001.08361 [cs.LG] <https://arxiv.org/abs/2001.08361>
- [20] Yi Li, Jieming Zhu, Weiwen Liu, Liangcai Su, Guohao Cai, Qi Zhang, Ruiming Tang, Xi Xiao, and Xiuqiang He. 2022. PEAR: Personalized Re-ranking with Contextualized Transformer for Recommendation. WWW.
- [21] Jianxun Lian, Xiaohuan Zhou, Fuzheng Zhang, Zhongxia Chen, Xing Xie, and Guangzhong Sun. 2018. xDeepFM: Combining Explicit and Implicit Feature Interactions for Recommender Systems. *KDD* (2018). <https://api.semanticscholar.org/CorpusID:3930042>
- [22] Kelong Mao, Jieming Zhu, Liangcai Su, Guohao Cai, Yuru Li, and Zhenhua Dong. 2023. FinalMLP: An Enhanced Two-Stream MLP Model for CTR Prediction. In *AAAI Conference on Artificial Intelligence*. <https://api.semanticscholar.org/CorpusID:257913572>
- [23] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmitry Dzhulgakov, Andrey Mallevich, Ilya Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. 2019. Deep Learning Recommendation Model for Personalization and Recommendation Systems. *CoRR abs/1906.00091* (2019). <https://arxiv.org/abs/1906.00091>
- [24] Liang Pang, Jun Xu, Qingyao Ai, Yanyan Lan, Xueqi Cheng, and Jirong Wen. 2020. SetRank: Learning a Permutation-Invariant Ranking Model for Information Retrieval. SIGIR.
- [25] Biswajit Paria, Chih-Kuan Yeh, Ian E. H. Yen, Ning Xu, Pradeep Ravikumar, and Barnabás Póczos. 2020. Minimizing FLOPs to Learn Efficient Sparse Representations. arXiv:2004.05665 [cs.LG] <https://arxiv.org/abs/2004.05665>
- [26] Changhua Pei, Yi Zhang, Yongfeng Zhang, Fei Sun, Xiao Lin, Hanxiao Sun, Jian Wu, Peng Jiang, Junfeng Ge, Wenwu Ou, and Dan Pei. 2019. Personalized re-ranking for recommendation. RecSys.
- [27] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. arXiv:2305.05065 [cs.LG] <https://arxiv.org/abs/2305.05065>
- [28] Yuxin Ren, Qiya Yang, Yichun Wu, Wei Xu, Yalong Wang, and Zhiqiang Zhang. 2024. Non-autoregressive Generative Models for Reranking Recommendation. KDD.
- [29] Anima Singh, Trung Vu, Nikhil Mehta, Raghunandan Keshavan, Maheswaran Sathiamoorthy, Yilin Zheng, Lichan Hong, Lukasz Heldt, Li Wei, Devansh Tandon, Ed H. Chi, and Xinyang Yi. 2024. Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations. arXiv:2306.08121 [cs.LG] <https://arxiv.org/abs/2306.08121>
- [30] Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. 2018. AutoInt: Automatic Feature Interaction Learning via Self-Attentive Neural Networks. *Proceedings of the 28th ACM International Conference on Information and Knowledge Management* (2018). <https://api.semanticscholar.org/CorpusID:53100214>
- [31] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [32] Qiang Wang, Bei Li, Tong Xiao, Jingbo Zhu, Changliang Li, Derek F Wong, and Lidia S Chao. 2019. Learning deep transformer models for machine translation. *ArXiv preprint arXiv:1906.01787* (2019).
- [33] Ruoxi Wang, Bin Fu, G. Fu, and Mingliang Wang. 2017. Deep & Cross Network for Ad Click Predictions. *Proceedings of the ADKDD'17* (2017). <https://api.semanticscholar.org/CorpusID:6011288>
- [34] Ruoxi Wang, Rakesh Shivanna, Derek Zhiyuan Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed H. Chi. 2020. DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems. *Proceedings of the Web Conference 2021* (2020). <https://api.semanticscholar.org/CorpusID:224845398>
- [35] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. 2021. SoundStream: An End-to-End Neural Audio Codec. arXiv:2107.03312 [cs.SD] <https://arxiv.org/abs/2107.03312>
- [36] Zhichen Zeng, Xiaolong Liu, Mengyue Hang, Xiaoyi Liu, Qinghai Zhou, Chaofei Yang, Yiqun Liu, Yichen Ruan, Laming Chen, Yuxin Chen, Yujia Hao, Jiaqi Xu, Jade Nie, Xi Liu, Buyun Zhang, Wei Wen, Siyang Yuan, Kai Wang, Wen-Yen Chen, Yiping Han, Huayu Li, Chunzhi Yang, Bo Long, Philip S. Yu, Hanghang Tong, and Jijian Yang. 2024. InterFormer: Towards Effective Heterogeneous Interaction Learning for Click-Through Rate Prediction. arXiv:2411.09852 [cs.LG] <https://arxiv.org/abs/2411.09852>
- [37] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, Yinghai Lu, and Yu Shi. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In *ICML*.
- [38] Buyun Zhang, Liang Luo, Yuxin Chen, Jade Nie, Xi Liu, Daifeng Guo, Yanli Zhao, Shen Li, Yuchen Hao, Yantao Yao, Guna Lakshminarayanan, Ellie Dingqiao Wen, Jongsoo Park, Maxim Naumov, and Wenlin Chen. 2024. Wukong: Towards a Scaling Law for Large-Scale Recommendation. arXiv:2403.02545 [cs.LG] <https://arxiv.org/abs/2403.02545>
- [39] Fengyu Zhang, Mohit Kothari, and Birjodh Tiwana. 2024. Leveraging Dwell Time to Improve Member Experiences on the LinkedIn Feed. <https://www.linkedin.com/blog/engineering/feed/leveraging-dwell-time-to-improve-member-experiences-on-the-linkedin-feed>

A APPENDIX

A.1 Reproducibility notes

Training stability emerged as a critical factor in optimizing our model architecture, and we implemented several techniques to ensure it. These included using a transformer architecture with gating, where layer normalization is applied before multi-head attention (MHA) or MLP layers, and gating applied on top of MHA/MLP outputs with a sigmoid(XW) layer. Additionally, we employed separate learning rates for embeddings (0.01) and dense layers (0.001) to further enhance stability. Architectural changes, while having limited direct impact on performance, were crucial for maintaining training stability, enabling effective scaling of model and input size — both of which proved more impactful for performance improvements.

Our exploration of features revealed that models with even a single feature could achieve performance near baseline production model levels. However, surpassing baseline performance required the inclusion of multiple features. Notably, we tracked performance on cold-start items—those with few or no interactions in the training data—and found that adding more features significantly boosted performance on these items, demonstrating the value of feature richness for addressing sparse data challenges.

Investigations into position embeddings showed no significant AUC difference between relative attention bias and learned position embeddings for history sequence, suggesting that this choice has minimal impact. Similarly, replacing MLPs with DCNv2 or incorporating DCNv2 at the input level did not result in performance gains, despite its theoretical potential to model interactions across embedding dimensions. Even removing MLPs from transformer blocks caused only minor performance drops, presenting an opportunity to trade off MLPs for additional MHA layers within the same memory budget.

Our experiments with alternative attention activation functions, such as SiLU and Sigmoid, also yielded subpar results compared to Softmax. While these activations were hypothesized to better model extreme affinity cases, Softmax’s normalization proved more effective at emphasizing the relative importance of sequence elements. This aligns with our findings on HSTU, which did not improve performance and showed lower AUC. Additionally, HSTU’s reliance on SiLU attention made it incompatible with FlashAttention, a critical component in larger-scale experiments.

Regarding vocabulary size, we found no AUC difference between item ID embedding vocabularies of 33 million and 66 million, despite the training data encompassing approximately 150 million unique object IDs. This suggests that a smaller vocabulary suffices without compromising performance.

A.2 Analysis of alternative solutions

We also explored alternative methods for scaling the model. Wukong [38] has demonstrated effective dense parameter scaling by allowing the model to capture higher-order feature interactions efficiently. However, directly applying Wukong to the Feed models

did not result in any observable improvement in AUCs, even when we scale the number of layers to 8. Our hypothesis suggests that the numerical features may have experienced information degradation during deep feature interactions with embedding features, potentially due to feature heterogeneity. The diverse nature of features, including numerical and categorical data, likely contributed to challenges in effectively capturing complex interactions.

We attempted to change the design for Wukong, where we employ a dual-pathway approach to feature processing to a baseline architecture (see §3.1). Embedding features are passed through a deep Wukong layer, facilitating deep vector-wise interactions. Concurrently, numerical and categorical features are fed into a two-layer DCNv2 to enable element-wise interactions. This bifurcated structure allows for specialized handling of different feature types, potentially mitigating the challenges posed by feature heterogeneity. With this updated architecture, we observed a 0.36% increase in Contributions AUC on top of baseline model (see §3.1), when scaling the number of layers to 8, which brings additional 2 million parameters. With a two-tower structure, we extend the scaling to accommodate multiple objectives. For example, our model can simultaneously optimize for various user interactions, such as clicks, comments, shares, and more. Additionally, we explore different mechanisms for concatenating embeddings to achieve optimal performance. It’s important to note that the Wukong layer is not restricted to stacked Factorization Machines; it can also incorporate stacked CrossNet for enhanced flexibility. The original Wukong Layer includes a Linear Compress Block, which linearly recombines embeddings — an important element for performance. When incorporated into the stacked CrossNet variant of Wukong, this block is adapted into a wide layer. This provides an interesting perspective, as each layer in Wukong effectively embodies the wide and deep architecture.

As we developed *LiGR*, our proposed architecture (Figure 2) achieved a 1.2% increase in Contributions AUC and demonstrated better scalability compared to the 0.36% improvement observed with the Wukong layer. We believe that Wukong could primarily be used to extend DLRM-style models [23] or to be used in combination with DLRM and transformer-based models, such as [36].